

CSS (Cascading Style Sheets)

1. What is CSS

CSS (Cascading Style Sheets) is a style sheet language used to design and control the appearance of web pages written in HTML. CSS defines how HTML elements should be displayed on screen, paper, or other media.

Why CSS is used

- Separates content (HTML) from design
- Makes websites attractive
- Saves time (one CSS file can style many pages)
- Responsive and device-friendly

Example

```
<p>This is a paragraph</p>
```

```
p {  
  color: blue;  
  font-size: 18px;  
}
```

2. Types of CSS

2.1 Inline CSS

CSS written directly inside an HTML tag.

Syntax

```
<h1 style="color:red; text-align:center;">Hello</h1>
```

Use: Small changes

Disadvantage: Hard to manage

2.2 Internal CSS

CSS written inside `<style>` tag in `<head>` section.

```
<head>
<style>
  p { color: green; }
</style>
</head>
```

2.3 External CSS

CSS written in a separate `.css` file.

```
<link rel="stylesheet" href="style.css">
```

```
body {
  background-color: lightgray;
}
```

Best practice

CSS2

3. Background

Properties

background-color

- **red** → sets background color to red
- **#fff** → sets background color to white
- **rgb(255,0,0)** → sets background color using RGB value (red)

background-image

- **url('img.jpg')** → sets an image as the background

background-repeat

- **repeat** → image repeats both horizontally and vertically (default)
- **no-repeat** → image appears only once
- **repeat-x** → image repeats only horizontally
- **repeat-y** → image repeats only vertically

background-position

- **left** → places image on the left side
- **right** → places image on the right side
- **center** → places image at the center
- **top** → places image at the top
- **bottom** → places image at the bottom

background-attachment

- **scroll** → background image scrolls with the page (default)
- **fixed** → background image stays fixed while page scrolls

Properties

- background-size: cover | contain

```
body {  
  background-size: cover;  
}  
body {  
  background-color: lightblue;  
  background-image: url('img.jpg');  
  background-repeat: no-repeat;  
}
```

4. Margin

Controls outer space

Properties:-

Margin

Sets space **outside an element on all sides**.

Example:

```
margin: 20px;  
→ 20px space on all sides
```

margin: top right bottom left;

Sets margin for **each side in one line (clockwise)**.

Order: Top → Right → Bottom → Left

Example:

```
margin: 10px 20px 30px 40px;
```

→ Top: 10px, Right: 20px, Bottom: 30px, Left: 40px

Margin-top

Sets space **above the element**.

Example:

```
margin-top: 10px;
```

Margin-right

Sets space on the **right side**.

Example:

```
margin-right: 15px;
```

Margin-bottom

Sets space **below the element**.

Example:

```
margin-bottom: 20px;
```

Margin-left

Sets space on the **left side**.

Example:

```
margin-left: 25px;
```

Values: px, %, auto

```
div {  
  margin: 20px;  
}
```

5. Padding

Controls inner space

Properties

padding

Sets space **inside an element**, between **content and border**.

Example:

```
padding: 20px;
```

→ 20px space on all sides inside the element

padding: top right bottom left;

Sets padding for **each side in one line (clockwise)**.

Order: Top → Right → Bottom → Left

Example:

```
padding: 10px 20px 30px 40px;
```

→ Top: 10px, Right: 20px, Bottom: 30px, Left: 40px

padding-top

Sets space **inside the top** of the element.

Example:

```
padding-top: 10px;
```

padding-right

Sets space **inside the right** of the element.

Example:

```
padding-right: 15px;
```

padding-bottom

Sets space **inside the bottom** of the element.

Example:

```
padding-bottom: 20px;
```

padding-left

Sets space **inside the left** of the element.

Example:

```
padding-left: 25px;
```

```
div {  
  padding: 15px;  
}
```

6. Border

Border is the line drawn **around an element**, between **padding and margin**.

It helps highlight elements and define boundaries.

Properties

- border-width: thin | medium | thick | px
- border-style: solid | dashed | dotted | double
- border-color: red | #000
- border: width border-style color;
 - border: 2px solid black;
- border-radius: 10px;

```
box {  
  border: 2px solid black;  
}
```

7. Positioning

CSS Positioning is used to **control the exact location of an element** on a web page. It works with **top, right, bottom, and left** properties.

. position: static

This is the **default position** of all elements. Elements appear in normal page flow.

- top, right, bottom, left **do not work**
- No special positioning

Example:

```
div {  
  position: static;  
}
```

2. position: relative

Element is positioned **relative to its normal position**.

- Original space is **kept**
- Can move using top, right, bottom, left

Example:

```
div {  
  position: relative;
```

```
top: 20px;
left: 10px;
}
```

➡ Moves 20px down and 10px right from its original position.

3. position: absolute

Element is positioned **relative to the nearest positioned parent** (or body if no parent is positioned).

- Removed from normal flow
- Does **not keep space**
- Moves freely

Example:

```
div {
  position: absolute;
  top: 0;
  right: 0;
}
```

➡ Moves to top-right corner.

4. position: fixed

Element is positioned **relative to the browser window**.

- Stays fixed while scrolling
- Removed from normal flow

Example:

```
div {
  position: fixed;
  bottom: 0;
  right: 0;
}
```

➡ Fixed at bottom-right of screen.

5. position: sticky

Mix of **relative** and **fixed**.

- Behaves like relative first
- Becomes fixed when scrolling reaches a point

Example:

```
div {  
  position: sticky;  
  top: 0;  
}
```

➡ Sticks to top during scroll.

Top, Right, Bottom, Left

Used to move positioned elements.

- `top: 10px;`
- `right: 20px;`
- `bottom: 15px;`
- `left: 25px;`

```
.box {  
  position: absolute;  
  top: 50px;  
}
```

8. Float

The `float` property is used to **move an element to the left or right** of its container.
Other content (like text) **wraps around the floated element**.

Values

- left
- right
- none

1. float: left

Moves the element to the **left side** of its container.

Example:

```
img {  
  float: left;  
}
```

➡ Text wraps on the right side of the image.

2. float: right

Moves the element to the **right side** of its container.

Example:

```
img {  
  float: right;  
}
```

➡ Text wraps on the left side of the image.

3. float: none

Default value.

Element does **not float**.

Example:

```
div {  
  float: none;  
}
```

clear Property

Used to **stop floating elements** from affecting other elements.

Values:

- left
- right

- both
- none

clear: left

Stops floating from the left.

```
div {  
  clear: left;  
}
```

clear: right

Stops floating from the right.

```
div {  
  clear: right;  
}
```

clear: both

Stops floating from **both sides**.

```
div {  
  clear: both;  
}
```

Float Layout Example

```
.box1 {  
  float: left;  
  width: 50%;  
}  
.box2 {  
  float: right;  
  width: 50%;  
}
```

 Two boxes side by side.

```
img {  
  float: left;  
}
```

9. Z-index

Controls overlapping

Values: number (1, 10, -1)

```
.box {  
  z-index: 10;  
}
```

10. Links

States

- a:link
- a:visited
- a:hover
- a:active

a:link

Styles a **normal, unvisited link**.

Example:

```
a:link {  
  color: blue;  
}
```

a:visited

Styles a **visited link**.

Example:

```
a:visited {  
  color: purple;  
}
```

a:hover

Styles a link when the **mouse is over it**.

Example:

```
a:hover {  
  color: red;  
}
```

a:active

Styles a link when it is **clicked**.

Example:

```
a:active {  
    color: green;  
}  
a:hover {  
    color: red;  
}
```

11. Text

Properties

- Color: red, pink, green
- text-align: left | right | center | justify
- text-decoration: none | underline | overline
- text-transform: uppercase | lowercase
- line-height: 2px

```
p {  
    text-align: justify;  
}
```

12. Outline

Properties

- outline-width: 2px;
- outline-style: solid;
- outline-color: red;

```
input {  
    outline: 2px solid blue;  
}
```

CSS3

13. Selectors

Types

- Element: p
- Class: .box
- ID: #main
- Group: h1, h2
- Attribute: input[type="text"]

```
.box { color: red; }
```

14. Text (CSS3)

Properties

- Text-shadow:
text-shadow is used to add a shadow effect behind text. It makes the text look more attractive or 3D.
 - text-shadow: x-offset y-offset blur-radius color;
 - text-shadow: 2px 2px 5px gray;
- word-wrap:
word-wrap is used to break long words so that they do not overflow outside the container.
 - word-wrap: break-word;

```
h1 {  
  text-shadow: 2px 2px 5px gray;  
}
```

15. Text Effects

- overflow: hidden | scroll
-

16. Box Shadow

```
div {  
  box-shadow: 5px 5px 10px black;  
}
```

17. 2D Transforms

Properties

◆ **translate()**

translate() moves an element from its original position.

✓ **Syntax**

```
transform: translate(x, y);
```

✓ **Example**

```
div {  
  transform: translate(50px, 20px);  
}
```

👉 Moves the element **50px to the right** and **20px down**.

You can also use:

```
transform: translateX(50px);  
transform: translateY(20px);
```

◆ **rotate()**

rotate() rotates an element clockwise or anti-clockwise.

✓ **Syntax**

```
transform: rotate(angle);
```

✓ **Example**

```
div {  
  transform: rotate(45deg);  
}
```

👉 Rotates the element **45 degrees clockwise**.

◆ **scale()**

scale() increases or decreases the size of an element.

✓ Syntax

```
transform: scale(x, y);
```

✓ Example

```
div {  
  transform: scale(1.5);  
}
```

👉 Makes the element **1.5 times bigger**.

Other options:

```
transform: scaleX(2);  
transform: scaleY(0.5);
```

skew()

skew() tilts (slants) an element along the X or Y axis.

✓ Syntax

```
transform: skew(x-angle, y-angle);
```

✓ Example

```
div {  
  transform: skew(20deg, 10deg);  
}
```

👉 Tilts the element **20° on X-axis** and **10° on Y-axis**.

You can also use:

```
transform: skewX(20deg);  
transform: skewY(10deg);
```

📌 Important Note

Multiple transforms can be combined:

```
transform: translate(50px, 0) rotate(30deg) scale(1.2);
div {
  transform: rotate(10deg);
}
```

18. 3D Transforms

Properties

◆ rotateX()

rotateX() rotates an element around the **X-axis** (horizontal axis).
It creates a **vertical flip** effect (top ↔ bottom).

✓ Syntax

```
transform: rotateX(angle);
```

✓ Example

```
div {
  transform: rotateX(60deg);
}
```

👉 The element tilts **forward or backward**.

◆ rotateY()

rotateY() rotates an element around the **Y-axis** (vertical axis).
It creates a **horizontal flip** effect (left ↔ right).

✓ Syntax

```
transform: rotateY(angle);
```

✓ Example

```
div {
  transform: rotateY(45deg);
}
```

👉 The element rotates **left or right in 3D space**.

◆ rotateZ()

rotateZ() rotates an element around the **Z-axis** (coming out of the screen).
This is the **same as rotate()** in 2D.

✓ Syntax

```
transform: rotateZ(angle);
```

✓ Example

```
div {  
  transform: rotateZ(90deg);  
}
```

👉 The element rotates **flat on the screen**.

📌 Important Notes

- To see proper 3D effect, use **perspective**:

```
.container {  
  perspective: 500px;  
}
```

- Combine rotations:

```
transform: rotateX(30deg) rotateY(45deg) rotateZ(20deg);  
div {  
  transform: rotateY(45deg);  
}
```

19. Transition

A **CSS transition** is used to make a change in CSS properties **smooth and gradual** instead of sudden.

It adds animation-like effects when an element changes from one state to another (for example, on hover).

Without transition, changes happen instantly.

With transition, changes happen slowly over time.

Properties

- transition-property

- transition-duration
- transition-timing-function
- transition-delay

♦ transition-property

Specifies **which CSS property** will change smoothly.

✓ Syntax

```
transition-property: property-name;
```

✓ Example

```
div {  
  transition-property: background-color;  
}
```

👉 Only `background-color` will animate.

You can also use:

```
transition-property: width, height;  
transition-property: all;
```

♦ transition-duration

Defines **how long the transition takes** to complete.

✓ Syntax

```
transition-duration: time;
```

✓ Example

```
div {  
  transition-duration: 2s;  
}
```

👉 Transition will complete in **2 seconds**.

♦ transition-timing-function

Controls **the speed curve** of the transition (how fast or slow it feels).

✓ Syntax

```
transition-timing-function: value;
```

✓ Common Values

Value	Meaning
ease	Slow start, fast middle, slow end (default)
linear	Same speed
ease-in	Slow start
ease-out	Slow end
ease-in-out	Slow start & end

✓ Example

```
div {  
  transition-timing-function: ease-in-out;  
}
```

♦ transition-delay

Sets a **delay** before the transition starts.

✓ Syntax

```
transition-delay: time;
```

✓ Example

```
div {  
  transition-delay: 1s;  
}
```

👉 Transition will start **after 1 second**.

♦ Combined Example

```
div {  
  width: 100px;  
  height: 100px;
```

```
background: blue;
transition-property: width;
transition-duration: 2s;
transition-timing-function: ease;
transition-delay: 0.5s;
}
div:hover {
  width: 200px;
}
```

Shortcut Property

```
transition: width 2s ease 0.5s;
```

Conclusion

CSS is the backbone of web design. CSS2 handles basic layout and styling, while CSS3 adds animations, effects, and modern design features.

20. CSS Animations

What is Animation in CSS?

CSS Animation is used to add movement and visual effects to web pages.

With animations:

- Text can move
- Colors can change
- Objects can rotate
- Images can bounce

Animations make websites more attractive and interactive.

Real-Life Example

 Just like cartoons show moving objects, CSS animations make webpage elements move.

Why Do We Use Animations?

Animations are used to:

- Make websites attractive
 - Grab user attention
 - Create smooth effects
 - Improve user experience
-

Main Parts of CSS Animation

CSS Animation has two main parts:

1. @keyframes

Defines how the animation will work.

2. Animation Properties

Control:

- speed
 - timing
 - duration
 - repetition
-

Syntax of CSS Animation

@keyframes animation_name

```
{  
  from  
  {  
    property: value;  
  }  
  
  to  
  {  
    property: value;  
  }  
}
```

Example 1: Simple Color Change Animation

HTML

```
<div class="box"></div>
```

CSS

```
.box
{
  width: 100px;
  height: 100px;
  background-color: red;

  animation-name: colorchange;
  animation-duration: 3s;
}

@keyframes colorchange
{
  from
  {
    background-color: red;
  }

  to
  {
    background-color: blue;
  }
}
```

Explanation

This animation:

- Starts with red color
- Changes to blue color
- Runs for 3 seconds

Animation Properties

Property	Work
animation-name	Name of animation
animation-duration	How long animation runs
animation-delay	Delay before animation starts
animation-iteration-count	Number of repetitions
animation-direction	Controls direction
animation-timing-function	Controls speed style

Example 2: Moving Box Animation

HTML

```
<div class="move"></div>
```

CSS

```
.move
{
  width: 100px;
  height: 100px;
  background-color: green;
  position: relative;

  animation-name: movingbox;
  animation-duration: 5s;
}
```

```
@keyframes movingbox
{
  from
  {
    left: 0px;
  }
}
```

```
to
{
  left: 300px;
}
```

Explanation

This animation:

- Moves the box from left to right
 - Completes movement in 5 seconds
-

Example 3: Infinite Animation

```
.box
{
  animation-iteration-count: infinite;
}
```

Explanation

The animation repeats forever.

Example 4: Rotate Animation

```
.box
{
  width: 100px;
  height: 100px;
  background: orange;

  animation: rotatebox 3s infinite;
}

@keyframes rotatebox
{
```

```
from
{
  transform: rotate(0deg);
}

to
{
  transform: rotate(360deg);
}
}
```

Explanation

This animation rotates the box completely.

Example 5: Bounce Effect

```
.ball
{
  width: 80px;
  height: 80px;
  background: blue;
  border-radius: 50%;
  position: relative;

  animation: bounce 1s infinite;
}
```

@keyframes bounce

```
{
  0%
  {
    top: 0px;
  }

  50%
  {
    top: 200px;
  }

  100%
```

```
{  
  top: 0px;  
}
```

Explanation

This animation makes the ball bounce up and down.

Animation Timing Functions

Value	Meaning
linear	Same speed throughout
ease	Slow start and end
ease-in	Slow start
ease-out	Slow end
ease-in-out	Slow start and end

Example

animation-timing-function: ease-in-out;

Shorthand Property

animation: movebox 3s infinite;

This single line combines multiple animation properties.

Advantages of CSS Animation

- Attractive design
 - Better user experience
 - Interactive webpages
 - Smooth visual effects
-

Disadvantages of CSS Animation

- Too many animations can slow the website
 - May distract users
 - Heavy animations reduce performance
-

Difference Between Transition and Animation

Transition	Animation
Needs a trigger	Can run automatically
Simple effects	Complex effects
One-time change	Multiple steps possible